Islamic University of Technology

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CSE 4802

SOFTWARE MAINTENANCE LAB

LAB PROJECT

Simulating the Four Categories of Software Maintenance, with Static/Dynamic Analysis and Program Slicing

Part 1: a case study on an existing production codebase (the *Jamia as-Suffah* Node.js / Express / MongoDB backend), applying Program Comprehension, Change Management, Impact Analysis, Reverse Engineering and Refactoring to one real, minimal example each of Corrective, Adaptive, Preventive and Perfective maintenance. **Part 2:** static analysis (AST generation, metrics, linting), dynamic analysis (code coverage), program slicing (static and dynamic), runtime tracing and profiling, and code clone detection applied to the same codebase.

SUBMITTED BY

MD Mainul Hasan

ID: 210042173

October 2026

Repository: https://github.com/notAvailable73/backend-Nodejs · branch lab/maintenance

Every figure in this report was regenerated from git tags by `npm run report` (Appendix A).

Contents

1. Introduction & Project Overview 3
Recovered architecture 3
2. Methodology 3
3. Tooling & Environment Evidence 5
Environment set-up as a first adaptation 5
Baseline test suite 6
4. Corrective Maintenance 7
1 · Program Comprehension 7
2 · Change Management 7
3 · Impact Analysis 7
4 · Reverse Engineering 8
5 · Refactoring (Implementation) 8
5. Adaptive Maintenance 10
1 · Program Comprehension 10
2 · Change Management 10
3 · Impact Analysis 10
4 · Reverse Engineering 11
5 · Refactoring (Implementation) 11
6. Preventive Maintenance 13
1 · Program Comprehension 13
2 · Change Management 13
3 · Impact Analysis 14
4 · Reverse Engineering 14
5 · Refactoring (Implementation) 14
7. Perfective Maintenance 17
1 · Program Comprehension 17
2 · Change Management 17
3 · Impact Analysis 17
4 · Reverse Engineering 17
5 · Refactoring (Implementation) 18
8. Summary & Conclusion 21
Residual findings 21
Conclusion 21
9. Tools Used & References 21
10. Overview & Methodology 23
11. Static Analysis A: AST Generation 24
12. Static Analysis B: Metrics Calculation 27
Lint findings 28
13. Dynamic Analysis: Code Coverage 29
14. Program Slicing (Static & Dynamic) 31
15. Runtime Analysis: Tracing & Profiling 33
CPU profiling 33
16. Code Clone Detection 36
17. Summary & Caveats 37
Appendix A. Reproduction 38
Evidence index 38

PART 1 · SOFTWARE MAINTENANCE

1. Introduction & Project Overview

SECTION 1

Subject system. The subject of this lab is the backend of *Jamia as-Suffah*, a website for an Islamic educational institution. It is a REST API written in Node.js with Express 4 and Mongoose 8 (MongoDB), deployed as a serverless function on Vercel. Users register and log in (the session is a JWT stored in an HTTP-only cookie); the main feature is a multilingual question-and-answer store in which each question carries English, Bengali and Arabic translations (`en` / `bn` / `ar`) and a list of tags, and can be filtered by tag or searched by title. A seminar model and a Cloudinary upload helper exist but are not yet wired to any route. At the start of this lab the code base had 17 JavaScript files and 432 source lines, 18 commits between January and March 2025, and no tests, README or design documents.

Why this system. It is small enough to understand completely, but it is a real project with real, unplanted defects: one malformed cookie can terminate the server, the server cannot start on current Node.js releases, passwords are written to the log, and error handling is copied by hand into each controller. Every maintenance activity below targets one of these, found by reading the code, running it, and reading its git history. The project's own last commits ("fix: add try catch block in login function", "refactor: comment out cookie setting in login response") show the developers already struggling with the same areas.

Recovered architecture

No architecture description exists, so the structure below was recovered from `src/` (Section 3 shows the file tree). Requests pass through four global middlewares, are routed to a controller, and the controller uses a Mongoose model. Errors are meant to reach a final error-handling middleware.



Recovered API contract

METHOD	PATH	AUTH	HANDLER	BEHAVIOUR
POST	`/api/auth/register`	—	registerUser	{name, email, password} → 201 {user, token}
POST	`/api/auth/login`	—	loginUser	{email, password} → 200 {user, token}, sets `token` cookie (7 days)
POST	`/api/auth/logout`	cookie	logoutUser	clears the cookie
GET	`/api/auth/profile`	cookie	getProfile	the logged-in user
GET	`/api/protected-route`	cookie	inline	"You are authenticated"
POST	`/api/questions/add`	—	createQuestion	{title, description, tags, language} → 201 question
GET	`/api/questions`	—	getQuestions	`?tags=a,b` and/or `?search=text` → 200 {count, questions} or 404

Table 1. Endpoints reconstructed from the route and controller files. Success responses are `{message, data}` (ApiResponse); error responses are `{success: false, message, errors}` (ApiError).

2. Methodology

SECTION 2

Software maintenance is conventionally divided into four categories (ISO/IEC/IEEE 14764): **Corrective** (repair a fault that causes failures), **Adaptive** (keep the software usable in a changed environment), **Preventive** (remove latent faults before they cause failures) and **Perfective** (improve maintainability or performance without changing behaviour). For each category, this lab made one deliberately small, self-contained change and documented it with the same five steps:

1. **Program Comprehension**: read and run the relevant code until the problem and its cause are understood.
2. **Change Management**: record a change request: id, category, risk, files touched, and the decision taken.
3. **Impact Analysis**: identify every file, caller and behaviour the change can affect.

4. **Reverse Engineering**: reconstruct the undocumented design knowledge (flows, contracts, dependency chains) the change relies on.
5. **Refactoring (Implementation)**: apply the minimal change and verify it with real tooling.

Test first. Before any change, a Jest + supertest suite was added that records the current behaviour. Every bug found during comprehension was written down as a `test.failing`: a test that states the *correct* behaviour and is expected to fail. Jest reports such a test as passing while the bug exists and as failing once the bug is gone, so each fix is proven by flipping its test to a normal `test`. For changes that add new expectations, the new tests are also run against the previous version's source to show that they fail there.

Traceability. Each change is one commit with its own git tag, so every "before" and "after" in this report is a real, checkable version of the project:

TAG	COMMIT	CONTENTS
`baseline`	`dfe8310`	the project exactly as found (origin/main)
`lab-setup`	`48c5b89`	+ Docker MongoDB, `.env.example`, Jest suite, README; `src/` unchanged
`cm-cor-01`	`fa4b34b`	+ Corrective change (Section 4)
`cm-ada-01`	`93ffde1`	+ Adaptive change (Section 5)
`cm-pre-01`	`b9a02ac`	+ Preventive change (Section 6)
`cm-per-01`	`b11cba5`	+ Perfective change (Section 7): the final version

Evidence. The dark terminal panes in this report are not screenshots. They are the real output of the commands shown in their first line, captured by `report/capture.mjs` from a fresh checkout of the tag named in the title bar, under the Node.js version shown. The caption under each pane names the evidence file it came from. Paths are shortened to `~/backend-Nodejs` and, where a pane is long, omitted lines are marked. Running `npm run report` regenerates every pane, number and the PDF itself (Appendix A).

3. Tooling & Environment Evidence

SECTION 3

Before any change, the project was installed and its starting state captured. The machine runs a recent Linux kernel and Node.js 26; MongoDB runs in Docker.

backend-Nodejs

```
$ uname -sr
Linux 7.0.0-38-generic

$ node --version && npm --version
v26.10.0
11.19.1

$ docker --version
Docker version 29.8.1, build 4a63305

$ docker exec jamiya-mongo mongod --version | head -1
db version v7.0.45
```

Environment used for every capture.

evidence/setup/env.txt · Node 26.10.0

lab-setup — node v26.10.0

```
$ npm ci
```

Installing the project's dependencies, required before the server or the tests can run.

evidence/setup/npm-ci.txt · tag lab-setup (48c5b89) · Node 26.10.0

baseline — node v26.10.0

```
$ git log --format='%h %ad %s' --date=short   # at tag baseline
dfe8310 2025-03-10 Feat: question can be fetched using tags and search
880a09a 2025-03-09 refactor: comment out cookie setting in login response
556f819 2025-03-09 feat: enhance login response with cookie SameSite attr
ibute and logging
536a680 2025-03-09 fix: add try catch block in login function
87870ad 2025-03-09 Merge branch 'main' of https://github.com/suffahUniver
sity/backend-Nodejs
bfd11e0 2025-03-09 feat: implement cookie-based authentication and update
API routes
fe19012 2025-02-19 Rename Question.model.js to question.model.js
bf87e0f 2025-02-19 fix: refactor server startup logic for vercel
8feae3a 2025-02-19 fix: update main entry point to src/server.js and enha
nce Vercel configuration
658ab10 2025-02-18 feat: add start script to package.json for production
use
d7ee724 2025-02-18 fix: change response method to send for welcome messag
e
a97c717 2025-02-18 vercel.json ./
6a0961e 2025-02-18 update Vercel rewrites to direct /src/server.js to /ap
i
c465f90 2025-02-15 add welcome message in root endpoint
00be4ff 2025-02-15 add Vercel configuration for API rewrites
f366673 2025-02-15 implement authentication routes
6b4457b 2025-01-15 added seminar model
b21247c 2025-01-12 initialize backend project with essential configuratio
ns and utilities
```

Commit history at the start of the lab (tag `baseline`).

evidence/setup/history.txt · tag baseline (dfe8310) · Node 26.10.0

baseline

```
$ git ls-tree -r --name-only baseline   # drawn as a tree
backend-Nodejs/
├── src/
│   ├── config/
│   │   └── database.js
│   ├── controllers/
│   │   ├── auth.controller.js
│   │   └── question.controller.js
│   ├── middlewares/
│   │   ├── auth.middleware.js
│   │   └── multer.middleware.js
│   ├── models/
│   │   ├── question.model.js
│   │   ├── seminar.model.js
│   │   └── user.model.js
│   ├── routes/
│   │   ├── auth.routes.js
│   │   ├── protected.routes.js
│   │   └── question.route.js
│   ├── utils/
│   │   ├── ApiError.js
│   │   ├── ApiResponse.js
│   │   ├── asyncHandler.js
│   │   └── cloudinary.js
│   ├── app.js
│   └── server.js
├── .gitignore
├── package-lock.json
├── package.json
└── vercel.json
```

Project structure at `baseline`.

evidence/setup/structure.txt · tag baseline (dfe8310)

Environment set-up as a first adaptation

The project expects a MongoDB server but provides no way to run one. A `docker-compose.yml` was added. The obvious image, MongoDB 8.0, refuses to start on this machine: Linux 6.19 and newer has a known incompatibility with MongoDB 8.x. MongoDB is therefore pinned to 7.0.45, which Mongoose 8.9 supports.

backend-Nodejs

```
$ docker run --rm mongo:8.0.34
{"t":{"$date":"2026-10-26T12:42:29.252Z"},"s":"F",  "c":"CONTROL",  "id":12257600,"ctx":"main","msg":"MongoDB cannot start: Linux kernel ver
sions 6.19 and newer has a known incompatibility with this version of MongoDB. See https://jira.mongodb.org/browse/SERVER-121912 for more in
formation."}
```

MongoDB 8.0 refuses to start on Linux 7.0 (only the fatal log line is kept).

evidence/setup/mongo8-kernel.txt · Node 26.10.0 · only the fatal log line is kept

Baseline test suite

The suite added in `lab-setup` runs against a separate `jamiya_test` database and does not change the application: `git diff baseline lab-setup -- src` is empty (evidence `setup/src-unchanged`). All 20 tests pass; 8 of them are `test.failing` tests that document known bugs (Table 2). Jest prints a `test.failing` test with a ✓ while the bug is still present.

lab-setup — node v24.21.0

```
$ npm test -- --verbose
… 3 line(s) above omitted
    √ stores the password hashed (75 ms)
    √ rejects a duplicate email with 400 (78 ms)
    √ does not expose the password hash (72 ms)
  POST /api/auth/login
    √ returns a JWT and sets it as an HTTP-only cookie (140 ms)
    √ rejects a wrong password with 401 (140 ms)
    √ rejects an unknown email with 401 (73 ms)
  GET /api/auth/profile
    √ returns the logged-in user without the password (142 ms)
    √ rejects a request without a token with 401 (1505 ms)
    √ handles an invalid token without rejecting (1 ms)
  POST /api/auth/logout
    √ clears the token cookie (145 ms)

 PASS  tests/questions.test.js
  POST /api/questions/add
    √ creates a question with one translation (46 ms)
    √ stores the tags on the question (11 ms)
    √ rejects a missing title with 400 (6 ms)
  GET /api/questions
    √ lists questions, newest first (15 ms)
    √ filters by tag (7 ms)
    √ searches titles case-insensitively (6 ms)
    √ answers 404 when nothing matches (3 ms)
    √ finds a question created through the API by its tag (6 ms)
    √ treats regex characters in the search term literally (6 ms)

Test Suites: 2 passed, 2 total
Tests:       20 passed, 20 total
Snapshots:   0 total
Time:        3.299 s, estimated 5 s
```

Baseline run of the suite (tag `lab-setup`, Node.js 24, the last release the original code starts on).

evidence/setup/tests.txt · tag lab-setup (48c5b89) · Node 24.21.0

KNOWN-BUG TEST (`test.failing`)	BUG IT RECORDS	RESOLVED BY
rejects a request without a token with 401	auth middleware never answers	CM-COR-01
handles an invalid token without rejecting	unhandled rejection terminates the process	CM-COR-01
does not expose the password hash	bcrypt hash returned to clients	CM-PRE-01
rejects a duplicate email with 400	error handler ignores ApiError status (500)	CM-PER-01
stores the tags on the question	tags nested inside the translation	residual (§8)
finds a question created through the API by its tag	consequence of the tags bug	residual (§8)
rejects a missing title with 400	`field?.trim() === ""` lets `undefined` through	residual (§8)
treats regex characters in the search term literally	search text passed to `$regex` unescaped	residual (§8)

Table 2. The eight known bugs recorded at `lab-setup`. Four are fixed by the maintenance changes; four are recorded as residual findings, outside this lab's one-change-per-category scope.

4. Corrective Maintenance

REPAIR A FAULT

1 · Program Comprehension

`isAuthenticated` (`src/middlewares/auth.middleware.js`) guards three routes. It reads the `token` cookie, verifies the JWT, loads the user and calls `next()`. Reading it next to Express 4's middleware contract shows two faults on the failure paths:

- **No cookie:** line 9 *returns* a new `ApiError` but never sends it. Express ignores a middleware's return value, so no response is ever written and no `next()` is called. The client waits until it times out.
- **Invalid or expired token:** `jwt.verify` throws, and the catch block on line 17 *throws* again. Because the middleware is an `async` function, this becomes a rejected promise. Express 4 does not look at promises returned by middleware (only Express 5 does), so the rejection is unhandled. Since Node.js 15 an unhandled rejection terminates the process.

The second fault is severe: a single request with any malformed or expired cookie stops the server for every user. Running the original code confirms both faults.

lab-setup — node v24.21.0

```
$ report/demos/auth-requests.sh
$ node src/server.js &

$ curl http://127.0.0.1:5055/api/auth/profile
curl: (28) Operation timed out after 3002 milliseconds with 0 bytes received

[no HTTP response]

$ curl --cookie token=not-a-jwt http://127.0.0.1:5055/api/auth/profile
curl: (52) Empty reply from server

[no HTTP response]

$ kill -0 $SERVER_PID   # is the server process still running?
no, it exited with code 1

── server output ──
✅ MongoDB connected successfully
🚀 MongoDB Connected!
🚀 Server is running at port: 5055

               Link: http://localhost:5055
file://~/backend-Nodejs/src/middlewares/auth.middleware.js:17
        throw new ApiError(401, "Unauthorized", "Invalid or expired token");
              ^

ApiError: Invalid or expired token
    at isAuthenticated (file://~/backend-Nodejs/src/middlewares/auth.middleware.js:17:15)
    at Layer.handle [as handle_request] (~/backend-Nodejs/node_modules/express/lib/router/layer.js:95:5)
    at next (~/backend-Nodejs/node_modules/express/lib/router/route.js:149:13)
    at Route.dispatch (~/backend-Nodejs/node_modules/express/lib/router/route.js:119:3)
  statusCode: 401,
  success: false,
  errors: 'Unauthorized'
}

Node.js v24.21.0
```

Before: the first request gets no response; the second terminates the server (exit code 1) with the `ApiError` from line 17 as the uncaught error.

evidence/cor/demo-before.txt · tag lab-setup (48c5b89) · Node 24.21.0

2 · Change Management

CHANGE ID	TYPE	RISK	FILES TOUCHED
CM-COR-01	**Corrective**	**Low**	**auth.middleware.js (1)**

Send both 401 responses with the existing `ApiError.send(res)` instead of returning or throwing them, and move `next()` out of the `try` block. The valid-token path is unchanged. Upgrading to Express 5 (which forwards rejected promises) was rejected for this change: it is a major-version migration with its own breaking changes, far larger than the fault.

3 · Impact Analysis

- **Direct:** the three routes that use the middleware, `POST /api/auth/logout`, `GET /api/auth/profile` and `GET /api/protected-route`. Requests with a valid token behave exactly as before.
- **Clients:** requests that used to time out or break the connection now receive 401 with the standard error body `{success, message, errors}`, the same shape as every other error the API returns.

- **Indirect:** with `next()` outside the `try`, an error thrown synchronously by a later handler is no longer caught and misreported as "Invalid or expired token"; it now reaches the error handler.
- **Residual:** a valid token for a deleted user still passes with `req.user = null`. This is recorded, not fixed here.

4 · Reverse Engineering

No description of the authentication flow exists, so it was reconstructed from `auth.controller.js` (where the cookie is issued) and the middleware (where it is checked):

```
POST /api/auth/login ── bcrypt.compare ok ──► jwt.sign({id, role}, JWT_SECRET, 7d) ──► Set-Cookie: token=…; HttpOnly

request to a protected route
   │
   ▼
token cookie present? ──no──► BEFORE: return new ApiError(…)   (never sent → request hangs)
   │                          AFTER:  401 "No token provided"
  yes
   ▼
jwt.verify(token) + User.findById(id) ──throws──► BEFORE: throw → unhandled rejection → process exits
   │                                              AFTER:  401 "Invalid or expired token"
  ok
   ▼
req.user = user (without password) ──► next() ──► route handler
```

5 · Refactoring (Implementation)

Before · src/middlewares/auth.middleware.js @ lab-setup (48c5b89)

```
 5 export const isAuthenticated = async (req, res, next) => {
 6     const token = req.cookies.token;
 7
 8     if (!token) {
 9         return new ApiError(401, "Unauthorized", "No token
   provided");
10     }
11
12     try {
13         const decoded = jwt.verify(token,
   process.env.JWT_SECRET);
14         req.user = await User.findById(decoded.id).select("-
   password");
15         next();
16     } catch (error) {
17         throw new ApiError(401, "Unauthorized", "Invalid or
   expired token");
18     }
19 };
```

After · src/middlewares/auth.middleware.js @ cm-cor-01 (fa4b34b)

```
 5 export const isAuthenticated = async (req, res, next) => {
 6     const token = req.cookies.token;
 7
 8     if (!token) {
 9         return new ApiError(401, "Unauthorized", "No token
   provided").send(res);
10     }
11
12     try {
13         const decoded = jwt.verify(token,
   process.env.JWT_SECRET);
14         req.user = await User.findById(decoded.id).select("-
   password");
15     } catch (error) {
16         return new ApiError(401, "Unauthorized", "Invalid or
   expired token").send(res);
17     }
18     next();
19 };
```

Changed lines are highlighted. Both failure paths now *send* their response.

backend-Nodejs

```
$ git diff lab-setup cm-cor-01 -- src
diff --git a/src/middlewares/auth.middleware.js b/src/middlewares/auth.middleware.js
index b6bbcf7..cfe920b 100644
--- a/src/middlewares/auth.middleware.js
+++ b/src/middlewares/auth.middleware.js
@@ -6,14 +6,14 @@ export const isAuthenticated = async (req, res, next) => {
     const token = req.cookies.token;

     if (!token) {
-        return new ApiError(401, "Unauthorized", "No token provided");
+        return new ApiError(401, "Unauthorized", "No token provided").send(res);
     }

     try {
         const decoded = jwt.verify(token, process.env.JWT_SECRET);
         req.user = await User.findById(decoded.id).select("-password");
-        next();
     } catch (error) {
-        throw new ApiError(401, "Unauthorized", "Invalid or expired token");
+        return new ApiError(401, "Unauthorized", "Invalid or expired token").send(res);
     }
+    next();
 };
\ No newline at end of file
```

The complete change as a git diff.

evidence/cor/diff.txt · Node 26.10.0

The two known-bug tests for this fault became normal tests. The invalid-token test, which previously had to call the middleware directly because the stray rejection would have broken the test run, now goes through HTTP and covers three kinds of bad token.

cm-cor-01 — node v24.21.0

```
$ npm test -- --verbose tests/auth.test.js -t 'GET /api/auth/profile'
 PASS  tests/auth.test.js
  POST /api/auth/register
  POST /api/auth/login
  GET /api/auth/profile
    √ returns the logged-in user without the password (208 ms)
    √ rejects a request without a token with 401 (3 ms)
    √ rejects a malformed token with 401 (4 ms)
    √ rejects a signed with another secret token with 401 (3 ms)
    √ rejects a expired token with 401 (3 ms)
  POST /api/auth/logout
Test Suites: 1 passed, 1 total
Tests:       8 skipped, 5 passed, 13 total
Snapshots:   0 total
Time:        0.626 s, estimated 2 s
```

After: the authentication tests (tag `cm-cor-01`).

evidence/cor/tests.txt · tag cm-cor-01 (fa4b34b) · Node 24.21.0

cm-cor-01 — node v24.21.0

```
$ report/demos/auth-requests.sh
$ node src/server.js &

$ curl http://127.0.0.1:5055/api/auth/profile
{
  "success": false,
  "message": "No token provided",
  "errors": "Unauthorized"
}
[HTTP 401]

$ curl --cookie token=not-a-jwt http://127.0.0.1:5055/api/auth/profile
{
  "success": false,
  "message": "Invalid or expired token",
  "errors": "Unauthorized"
}
[HTTP 401]

$ kill -0 $SERVER_PID   # is the server process still running?
yes, still running

── server output ──
✅ MongoDB connected successfully
🚀 MongoDB Connected!
🚀 Server is running at port: 5055

               Link: http://localhost:5055
```

After: the same requests as before receive 401 responses and the server keeps running.

evidence/cor/demo-after.txt · tag cm-cor-01 (fa4b34b) · Node 24.21.0

Result. A fault that let any client stop the server with one request is removed by a 3-line change in one file. The suite grows from 20 to 22 tests, all passing, and known-bug tests drop from 8 to 6.

5. Adaptive Maintenance

ADJUST TO A CHANGED ENVIRONMENT

1 · Program Comprehension

The project's environment moved on without it. Node.js 25 removed `SlowBuffer`, an API deprecated for years (DEP0030), and current Node.js releases (this machine runs 26) no longer have it. Starting the unmodified server under Node.js 26 fails before a single line of application code runs:

```
cm-cor-01 — node v26.10.0

$ report/demos/boot.sh
$ node --version
v26.10.0

$ node src/server.js
~/backend-Nodejs/node_modules/buffer-equal-constant-time/index.js:37
var origSlowBufEqual = SlowBuffer.prototype.equal;
                               ^


TypeError: Cannot read properties of undefined (reading 'prototype')
    at Object.<anonymous> (~/backend-Nodejs/node_modules/buffer-equal-constant-time/index.js:37:35)
    at Module._compile (node:internal/modules/cjs/loader:1956:14)
    at Object..js (node:internal/modules/cjs/loader:2096:10)
    at Module.load (node:internal/modules/cjs/loader:1678:32)
    at Module._load (node:internal/modules/cjs/loader:1470:12)
    at wrapModuleLoad (node:internal/modules/cjs/loader:261:19)
    at Module.require (node:internal/modules/cjs/loader:1701:12)
    at require (node:internal/modules/helpers:318:16)
    at Object.<anonymous> (~/backend-Nodejs/node_modules/jwa/index.js:1:19)
    at Module._compile (node:internal/modules/cjs/loader:1956:14)

Node.js v26.10.0
[exited with code 1]
```

Before: the server cannot start on Node.js 26.

evidence/ada/boot-before.txt · tag cm-cor-01 (fa4b34b) · Node 26.10.0

The stack trace leads into `node_modules`, not `src/`. The application never uses `SlowBuffer` itself; a transitive dependency of `jsonwebtoken` does, at import time:

```
cm-cor-01 — node v26.10.0

$ npm ls buffer-equal-constant-time
jamiya-backend@1.0.0 ~/backend-Nodejs
└─┬ jsonwebtoken@9.0.2
  └─┬ jws@3.2.2
    └─┬ jwa@1.4.1
      └── buffer-equal-constant-time@1.0.1
```

The dependency chain that loads the old package.

evidence/ada/deps-before.txt · tag cm-cor-01 (fa4b34b) · Node 26.10.0

```
cm-cor-01 — node v26.10.0

$ grep -n SlowBuffer node_modules/buffer-equal-constant-time/index.js
4:var SlowBuffer = require('buffer').SlowBuffer;
31:  Buffer.prototype.equal = SlowBuffer.prototype.equal = function equal
(that) {
37:var origSlowBufEqual = SlowBuffer.prototype.equal;
40:  SlowBuffer.prototype.equal = origSlowBufEqual;
```

The package reads `SlowBuffer.prototype` when it is loaded.

evidence/ada/slowbuffer.txt · tag cm-cor-01 (fa4b34b) · Node 26.10.0

This is adaptive maintenance in its purest form: the code did not change and was not wrong when written. The runtime it depends on changed underneath it.

2 · Change Management

CHANGE ID	TYPE	RISK	FILES TOUCHED
CM-ADA-01	**Adaptive**	**Low**	**package.json, package-lock.json, .nvmrc, README.md (4)**

Three options were considered:

OPTION	EFFECT	DECISION
Pin Node.js 24 permanently	works today; the problem returns when Node.js 24 reaches end of life	rejected
Replace `jsonwebtoken` with `jose`	modern, no legacy dependencies, but async API: changes the User model, the middleware and both controllers that issue tokens	rejected (too large)
Upgrade `jsonwebtoken` 9.0.2 → 9.0.3	patch release; pulls in `jws` 4 / `jwa` 2, same API	**chosen**

3 · Impact Analysis

- **Direct:** the two places that use `jsonwebtoken`: token signing in `User.getJwtToken` and verification in `isAuthenticated`. Both use `sign` / `verify`, whose API is unchanged in 9.0.3.

- **Data:** tokens are still HS256 JWTs signed with the same secret, so sessions issued before the upgrade stay valid.
- **Environment:** `.nvmrc` moves to Node.js 26.10.0 and `engines` becomes `⩾20`. The deployment's Node.js setting (Vercel) should be raised to match.
- **Security side effect:** `npm audit` drops from 9 to 8 vulnerabilities, because the old `jws` 3 is gone.

4 · Reverse Engineering

The fix only works if the new chain does not load the old package. `npm ls` shows that `buffer-equal-constant-time` is *still installed* after the upgrade, which at first looks like no fix at all. Reading `jwa` 2.0.1 explains it: the old package is only `require`d inside a fallback for runtimes without `crypto.timingSafeEqual`, which every supported Node.js version has.

cm-ada-01 — node v26.10.0

```
$ npm ls buffer-equal-constant-time
jamiya-backend@1.0.0 ~/backend-Nodejs
└─┬ jsonwebtoken@9.0.3
  └─┬ jws@4.0.1
    └─┬ jwa@2.0.1
      └── buffer-equal-constant-time@1.0.1
```

After: the old package is still in the tree...

evidence/ada/deps-after.txt · tag cm-ada-01 (93ffde1) · Node 26.10.0

cm-ada-01 — node v26.10.0

```
$ grep -n -A9 '^var timingSafeEqual' node_modules/jwa/index.js
138:var timingSafeEqual = 'timingSafeEqual' in crypto ? function timingSa
feEqual(a, b) {
139-  if (a.byteLength !== b.byteLength) {
140-    return false;
141-  }
142-
143-  return crypto.timingSafeEqual(a, b)
144-} : function timingSafeEqual(a, b) {
145-  if (!bufferEqual) {
146-    bufferEqual = require('buffer-equal-constant-time');
147-  }
```

...but `jwa` 2 loads it only when `crypto.timingSafeEqual` is missing.

evidence/ada/jwa.txt · tag cm-ada-01 (93ffde1) · Node 26.10.0

```
BEFORE  jsonwebtoken 9.0.2 → jws 3.2.2 → jwa 1.4.1 ──require at load──► buffer-equal-constant-time ─► SlowBuffer.prototype → TypeError
AFTER   jsonwebtoken 9.0.3 → jws 4.0.1 → jwa 2.0.1 ──uses──► crypto.timingSafeEqual   (old package: lazy fallback, never loaded)
```

5 · Refactoring (Implementation)

backend-Nodejs

```
diff --git a/.nvmrc b/.nvmrc
index df6ae33..8a94b99 100644
--- a/.nvmrc
+++ b/.nvmrc
@@ -1 +1 @@
-24.21.0
+26.10.0
diff --git a/package.json b/package.json
index 0fa52df..9d2181b 100644
--- a/package.json
+++ b/package.json
@@ -12,7 +12,7 @@
     "db:down": "docker compose down"
   },
   "engines": {
-    "node": "⩾20 <25"
+    "node": "⩾20"
   },
   "repository": {
     "type": "git",
@@ -31,7 +31,7 @@
     "cors": "^2.8.5",
     "dotenv": "^16.4.7",
     "express": "^4.21.2",
-    "jsonwebtoken": "^9.0.2",
+    "jsonwebtoken": "^9.0.3",
     "mongoose": "^8.9.4",
     "multer": "^1.4.5-lts.1"
   },
 .nvmrc            |  2 +-
 README.md         |  4 +--
 package-lock.json | 29 ++++++++++++++++++----------
 package.json      |  4 ++--
 4 files changed, 20 insertions(+), 19 deletions(-)
```

The change: one dependency version, the Node.js pin and the engines range. The lock file changes only the three packages of the JWT chain.

evidence/ada/diff.txt · Node 26.10.0

cm-ada-01 — node v26.10.0

```
$ report/demos/boot.sh
$ node --version
v26.10.0

$ node src/server.js
✅ MongoDB connected successfully
🚀 MongoDB Connected!
🚀 Server is running at port: 5055

               Link: http://localhost:5055
[still running after 6 s, stopped by the demo]
```

After: the server starts on Node.js 26.

evidence/ada/boot-after.txt · tag cm-ada-01 (93ffde1) · Node 26.10.0

cm-ada-01 — node v26.10.0

```
$ node --version && npm test
v26.10.0
 PASS  tests/auth.test.js
 PASS  tests/questions.test.js
Test Suites: 2 passed, 2 total
Tests:       22 passed, 22 total
Snapshots:   0 total
Time:        1.793 s, estimated 2 s
```

Full suite on Node.js 26.

evidence/ada/tests-node26.txt · tag cm-ada-01 (93ffde1) · Node 26.10.0

cm-ada-01 — node v24.21.0

```
$ node --version && npm test
v24.21.0
 PASS  tests/auth.test.js
 PASS  tests/questions.test.js
Test Suites: 2 passed, 2 total
Tests:       22 passed, 22 total
Snapshots:   0 total
Time:        1.775 s, estimated 2 s
```

Full suite still passes on Node.js 24.

evidence/ada/tests-node24.txt · tag cm-ada-01 (93ffde1) · Node 24.21.0

Result. The application runs on current Node.js releases again and keeps working on Node.js 24. The change touches no application code, and all 22 tests pass on both runtimes.

6. Preventive Maintenance

FIX LATENT FAULTS BEFORE THEY CAUSE FAILURES

1 · Program Comprehension

Following the password through the authentication code reveals three latent faults. None has caused a visible failure yet, which is what makes this preventive rather than corrective work, but each one leaks a credential:

- `registerUser` starts with `console.log(req.body)`, writing every new user's **plaintext password** to the server log (on Vercel, readable by anyone with access to the project's logs).
- `loginUser` logs `JSON.stringify({user, token})`: the **bcrypt hash** and a valid **session token**.
- Both endpoints return the full user document, **including the bcrypt hash**, in the response body. A leaked hash can be attacked offline without any rate limit.

cm-ada-01 — node v26.10.0

```
$ report/demos/credentials.sh
$ node src/server.js &

$ curl -X POST http://127.0.0.1:5055/api/auth/register -d '{"name":"Demo User","email":"demo@example.com","password":"secret123"}'
{
  "message": "User registered successfully",
  "data": {
    "user": {
      "name": "Demo User",
      "email": "demo@example.com",
      "password": "$2b$10$r3NjP9OZPqP/cGRjkE9vb.GPbcTbVbEchsCdbl1EbT.0lw8L.8c3u",
      "role": "user",
      "askedQuestions": [],
      "_id": "6adf4ad0047a1be11a7518e3",
      "createdAt": "2026-10-26T12:42:57.003Z",
      "updatedAt": "2026-10-26T12:42:57.003Z",
      "__v": 0
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhZGY0YWQwMDQ3YTFiZTExYTc1MThlMyIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNzkzMDE4NTc3LCJleHAiOjE3OTM2MjMzNzd9.X8YqrAxkMRrdTB2Qyg861J0cHAVdA3dxgRlPyTit3h4"
  }
}
[HTTP 201]

$ curl -X POST http://127.0.0.1:5055/api/auth/login -d '{"email":"demo@example.com","password":"secret123"}'
{
  "message": "Login successful",
  "data": {
    "user": {
      "_id": "6adf4ad0047a1be11a7518e3",
      "name": "Demo User",
      "email": "demo@example.com",
      "password": "$2b$10$r3NjP9OZPqP/cGRjkE9vb.GPbcTbVbEchsCdbl1EbT.0lw8L.8c3u",
      "role": "user",
      "askedQuestions": [],
      "createdAt": "2026-10-26T12:42:57.003Z",
      "updatedAt": "2026-10-26T12:42:57.003Z",
      "__v": 0
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhZGY0YWQwMDQ3YTFiZTExYTc1MThlMyIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNzkzMDE4NTc3LCJleHAiOjE3OTM2MjMzNzd9.X8YqrAxkMRrdTB2Qyg861J0cHAVdA3dxgRlPyTit3h4"
  }
}
[HTTP 200]

── server output ──
✅ MongoDB connected successfully
🚀 MongoDB Connected!
🚀 Server is running at port: 5055

              Link: http://localhost:5055
… 16 line(s) below omitted
```

Before: the hash appears in both responses; the server log contains the plaintext password, the hash and the token (log truncated).

evidence/pre/demo-before.txt · tag cm-ada-01 (93ffde1) · Node 26.10.0

2 · Change Management

CHANGE ID	TYPE	RISK	FILES TOUCHED
CM-PRE-01	**Preventive**	**Low**	**user.model.js, auth.controller.js (2)**

Remove the two logging statements that print credentials, and strip `password` when a User document is converted to JSON, in the schema itself. Deleting the field in each controller was rejected: it would fix today's two routes but not the next route that returns a user. A schema-level rule protects routes that do not exist yet, which is the point of preventive maintenance.

3 · Impact Analysis

- **Direct:** every response that serialises a User: register, login, profile and the protected route.
- **Unaffected:** storage and authentication. `comparePassword` reads the field from the document, not from JSON, and the pre-save hashing hook is untouched.
- **Clients:** no legitimate client needs the hash; the token is still returned and set as a cookie.
- **Operations:** the server log loses the request-body line. `loginUser` still logs the email address (personal data, recorded as residual).

4 · Reverse Engineering

A source-to-sink data-flow map of the password, recovered from the controller, the model's hooks and Express's JSON serialisation:



5 · Refactoring (Implementation)

backend-Nodejs

```
$ git diff cm-ada-01 cm-pre-01 -- src
diff --git a/src/controllers/auth.controller.js b/src/controllers/auth.controller.js
index c1e745a..cfd7af2 100644
--- a/src/controllers/auth.controller.js
+++ b/src/controllers/auth.controller.js
@@ -4,7 +4,6 @@ import { ApiError } from "../utils/ApiError.js";
 import { User } from "../models/user.model.js";

 const registerUser = asyncHandler(async (req, res) => {
-    console.log(req.body);
     const { name, email, password } = req.body;

     if ([name, email, password].some(field => field?.trim() === "")) {
@@ -48,7 +47,6 @@ const loginUser = asyncHandler(async (req, res) => {
             secure: true,
             maxAge: 7 * 24 * 60 * 60 * 1000, // 7 days
         });
-        console.log("Sending response:", JSON.stringify({ user, token }, null, 2));
         return new ApiResponse(200, { user, token }, "Login successful").send(res);

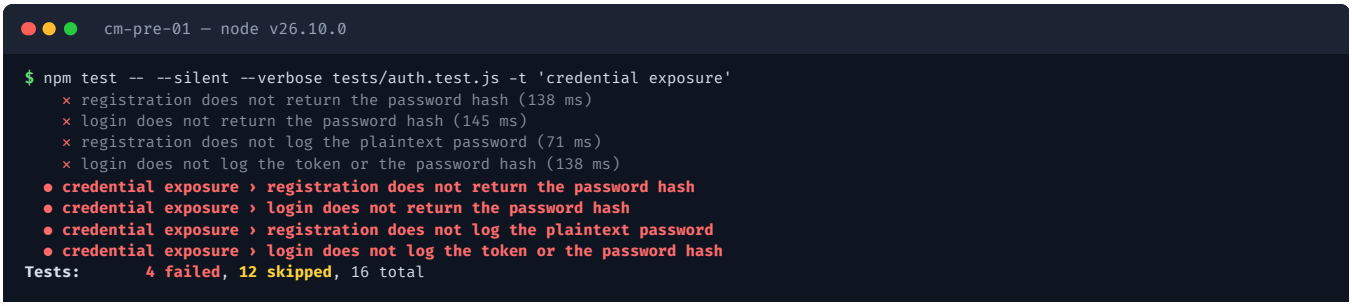
     } catch (error) {
diff --git a/src/models/user.model.js b/src/models/user.model.js
index ad2475f..0417f77 100644
--- a/src/models/user.model.js
+++ b/src/models/user.model.js
@@ -33,6 +33,14 @@ const userSchema = new mongoose.Schema({
     timestamps: true
 });

+// Never serialize the password hash, whichever route returns a user
+userSchema.set("toJSON", {
+    transform: (doc, ret) => {
+        delete ret.password;
+        return ret;
+    },
+});
+
 // Hash password before saving user
 userSchema.pre("save", async function (next) {
     if (!this.isModified("password")) return next();
```

The change as a git diff.

evidence/pre/diff.txt · Node 26.10.0

Four tests were written for the new rule. Run against the previous version's source, all four fail; with the change, all pass:



Before: the new tests against the source of `cm-ada-01` (failure details omitted).

evidence/pre/tests-before.txt · tag cm-pre-01 (b9a02ac) · Node 26.10.0 · tests from cm-pre-01, application source (src/) from cm-ada-01



After: the same tests on `cm-pre-01`.

evidence/pre/tests-after.txt · tag cm-pre-01 (b9a02ac) · Node 26.10.0

cm-pre-01 — node v26.10.0

```
$ report/demos/credentials.sh
$ node src/server.js &

$ curl -X POST http://127.0.0.1:5055/api/auth/register -d '{"name":"Demo User","email":"demo@example.com","password":"secret123"}'
{
  "message": "User registered successfully",
  "data": {
    "user": {
      "name": "Demo User",
      "email": "demo@example.com",
      "role": "user",
      "askedQuestions": [],
      "_id": "6adf4ad3e6077970efbf1892",
      "createdAt": "2026-10-26T12:42:59.986Z",
      "updatedAt": "2026-10-26T12:42:59.986Z",
      "__v": 0
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhZGY0YWQzZTYwNzc5NzBlZmJmMTg5MiIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNzkzMDE4NTgwLCJleHAiOjE3OTM2MjMzODB9.Z0tSnic80uA-Xsn9VW1pWE7mROr-FxvUfYl2QtNkLlo"
  }
}
[HTTP 201]

$ curl -X POST http://127.0.0.1:5055/api/auth/login -d '{"email":"demo@example.com","password":"secret123"}'
{
  "message": "Login successful",
  "data": {
    "user": {
      "_id": "6adf4ad3e6077970efbf1892",
      "name": "Demo User",
      "email": "demo@example.com",
      "role": "user",
      "askedQuestions": [],
      "createdAt": "2026-10-26T12:42:59.986Z",
      "updatedAt": "2026-10-26T12:42:59.986Z",
      "__v": 0
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhZGY0YWQzZTYwNzc5NzBlZmJmMTg5MiIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNzkzMDE4NTgwLCJleHAiOjE3OTM2MjMzODB9.Z0tSnic80uA-Xsn9VW1pWE7mROr-FxvUfYl2QtNkLlo"
  }
}
[HTTP 200]

── server output ──
✅ MongoDB connected successfully
🚀 MongoDB Connected!
🚀 Server is running at port: 5055

                Link: http://localhost:5055
demo@example.com
```

After: no hash in either response; the server log holds no password, hash or token.

evidence/pre/demo-after.txt · tag cm-pre-01 (b9a02ac) · Node 26.10.0

Result. Three credential leaks are closed, and the schema rule covers every future route that returns a user. Tests: 25, all passing; known-bug tests: 5.

7. Perfective Maintenance

IMPROVE STRUCTURE WITHOUT CHANGING BEHAVIOUR

1 · Program Comprehension

The code has two competing error-handling mechanisms. `asyncHandler` forwards any rejected promise to the global error handler in `app.js`, but that handler ignores `ApiError` and always answers 500. To get the right status codes anyway, three controllers wrap their entire body in the same hand-written `try/catch` that rebuilds the error and sends it. The clone detector finds the copies:

```
cm-pre-01 — node v26.10.0

$ jscpd --min-lines 5 --min-tokens 30 src
Duplications detection: Found 3 exact clones with 23(4.28%) duplicated lines in 17 (1 formats) files.
 - controllers/auth.controller.js [50:72 - 57:27] (8 lines, 42 tokens)
   controllers/question.controller.js [26:5 - 32:23]
 - controllers/auth.controller.js [50:72 - 57:27] (8 lines, 42 tokens)
   controllers/question.controller.js [79:5 - 86:23]
 - models/question.model.js [3:33 - 9:7] (7 lines, 33 tokens)
   models/seminar.model.js [3:32 - 9:7]
```

Before: two clone pairs of the 8-line catch block (and one acceptable clone in the models, discussed in §16).
evidence/per/jscpd-before.txt · tag cm-pre-01 (b9a02ac) · Node 26.10.0

The duplication is a design fault, not just untidiness: correct behaviour depends on remembering to copy the block. `registerUser` has no copy, so its "user already exists" error, an `ApiError(400)`, reaches the global handler and becomes a 500. `getQuestions` passes its 404 message in the wrong argument and relies on the catch block's fallback text.

2 · Change Management

CHANGE ID	TYPE	RISK	FILES TOUCHED
CM-PER-01	**Perfective**	**Medium (every error path)**	**app.js, auth.controller.js, question.controller.js (3)**

Make the global handler the single place where errors become responses: it sends an `ApiError` with its own status and body, and answers anything else with a generic 500 in the same shape. Then delete the three copied `try/catch` blocks, since `asyncHandler` already forwards errors. Fix the argument order of the one 404 that relied on the old fallback.

3 · Impact Analysis

Every error path in the API is affected, so each was listed and checked:

ERROR PATH	BEFORE	AFTER
login: missing fields / wrong credentials	400 / 401 via local catch	same status and body via global handler
question: empty field	400 via local catch	same
register: duplicate email	500 `{message: "Something went wrong!"}`	400 `{success: false, message: "A user with this email already exists", errors: …}`
questions: nothing found	404 "Error occurred while fetching questions"	404 "No questions found" / "No questions found matching …"
unexpected error (e.g. Mongoose validation)	500 with the internal error message and object	500 `{success: false, message: "Something went wrong!", errors: []}`; stack trace logged on the server

Success paths are unchanged (covered by the existing tests). The last row is a deliberate behaviour change with a security benefit: internal error details no longer reach clients.

4 · Reverse Engineering



5 · Refactoring (Implementation)

Before · src/app.js @ cm-pre-01 (b9a02ac)

```
21 // Error Handling Middleware
22 app.use((err, req, res, next) => {
23     console.error(err.stack);
24     res.status(500).json({ message: 'Something went wrong!' });
25 });
26
27 export {app}
```

After · src/app.js @ cm-per-01 (b11cba5)

```
22 // Error Handling Middleware: the single place where errors
     become responses
23 app.use((err, req, res, next) => {
24     if (err instanceof ApiError) {
25         return err.send(res);
26     }
27     console.error(err.stack);
28     new ApiError(500, [], 'Something went wrong!').send(res);
29 });
30
31 export {app}
```

The global error handler, before and after.

backend-Nodejs

```
$ git diff --stat cm-pre-01 cm-per-01 -- src
 src/app.js                          |   8 ++-
 src/controllers/auth.controller.js  |  56 ++++++----------
 src/controllers/question.controller.js | 119 +++++++++++++++----------------
 3 files changed, 77 insertions(+), 106 deletions(-)
```

Size of the change: the three controllers lose their copied blocks.

evidence/per/diff-stat.txt · Node 26.10.0

backend-Nodejs

```
$ git diff -w cm-pre-01 cm-per-01 -- src
diff --git a/src/app.js b/src/app.js
index a945789..e77cbe8 100644
--- a/src/app.js
+++ b/src/app.js
@@ -1,6 +1,7 @@
 import express from "express"
 import cors from "cors"
 import cookieParser from "cookie-parser";
+import { ApiError } from "./utils/ApiError.js";
 const app = express();
 app.use(cors());
 app.use(express.json());
@@ -18,10 +19,13 @@ import protectedRoute from "./routes/protected.routes.js";
 app.use("/api/questions", QuestionRoute);
 app.use("/api/auth", authRoute);
 app.use("/api/protected-route", protectedRoute);
-// Error Handling Middleware
+// Error Handling Middleware: the single place where errors become responses
 app.use((err, req, res, next) => {
+    if (err instanceof ApiError) {
+        return err.send(res);
+    }
     console.error(err.stack);
-    res.status(500).json({ message: 'Something went wrong!' });
+    new ApiError(500, [], 'Something went wrong!').send(res);
 });

 export {app}
\ No newline at end of file
diff --git a/src/controllers/auth.controller.js b/src/controllers/auth.controller.js
index cfd7af2..8cd52b4 100644
--- a/src/controllers/auth.controller.js
+++ b/src/controllers/auth.controller.js
@@ -24,7 +24,6 @@ const registerUser = asyncHandler(async (req, res) => {
 });

 const loginUser = asyncHandler(async (req, res) => {
-    try {
     const { email, password } = req.body;
     console.log(email);
     if ([email, password].some(field => field?.trim() === "")) {
@@ -38,7 +37,6 @@ const loginUser = asyncHandler(async (req, res) => {

     const token = user.getJwtToken();

-
     // Set token in HTTP-only cookie
     res.cookie("token", token, {
         httpOnly: true,
@@ -48,16 +46,6 @@ const loginUser = asyncHandler(async (req, res) => {
     });
     return new ApiResponse(200, { user, token }, "Login successful").send(res);
-
-    } catch (error) {
-        console.log(error);
-        const errorResponse = new ApiError(
-          error.statusCode || 500,
-          error.errors || error,
-          error.message || "Error occured while logging in"
-        );
-        errorResponse.send(res);
-     }
 });

 const logoutUser = asyncHandler(async (req, res) => {
diff --git a/src/controllers/question.controller.js b/src/controllers/question.controller.js
index 870c24e..30e831b 100644
--- a/src/controllers/question.controller.js
+++ b/src/controllers/question.controller.js
@@ -4,7 +4,6 @@ import { ApiResponse } from "../utils/ApiResponse.js";
 import { asyncHandler } from "../utils/asyncHandler.js";

 const createQuestion = asyncHandler(async (req, res) => {
-  try {
   const { title, description, tags, language } = req.body;
   if (
     [title, description, tags, language].some((field) => field?.trim() === "")
@@ -24,21 +23,11 @@ const createQuestion = asyncHandler(async (req, res) => {
     savedQuestion,
… 48 line(s) below omitted
```

The change, ignoring whitespace (`-w`): removing each `try` re-indents the whole function, which would otherwise hide the real edits.

evidence/per/diff.txt · Node 26.10.0

```
cm-per-01 — node v26.10.0

$ jscpd --min-lines 5 --min-tokens 30 src
Duplications detection: Found 1 exact clones with 7(1.38%) duplicated lines in 17 (1 formats) files.
 - models/question.model.js [3:33 - 9:7] (7 lines, 33 tokens)
   models/seminar.model.js [3:32 - 9:7]
```

After: the catch-block clones are gone; only the model clone remains.

evidence/per/jscpd-after.txt · tag cm-per-01 (b11cba5) · Node 26.10.0

```
cm-per-01 — node v26.10.0

$ npm test -- --silent
  ● POST /api/auth/register › rejects a duplicate email with 400
  ● GET /api/questions › answers 404 when nothing matches
  ● GET /api/questions › names the search term in the 404 message
Tests:       3 failed, 23 passed, 26 total
```

Before: the suite of `cm-per-01` run against the source of `cm-pre-01`. Three expectations fail: the duplicate-email status and the two 404 messages.

evidence/per/tests-before.txt · tag cm-per-01 (b11cba5) · Node 26.10.0 · tests from cm-per-01, application source (src/) from cm-pre-01

```
cm-per-01 — node v26.10.0

$ npm test
 PASS  tests/auth.test.js
 PASS  tests/questions.test.js
Test Suites: 2 passed, 2 total
Tests:       26 passed, 26 total
Snapshots:   0 total
Time:        2.15 s, estimated 3 s
```

After: all tests pass.

evidence/per/tests-after.txt · tag cm-per-01 (b11cba5) · Node 26.10.0

Result. Error handling now lives in one place. Clones drop from 3 to 1; the cyclomatic complexity of `getQuestions` falls from 11 to 7 and of `loginUser` from 8 to 4 (§12). One known bug is fixed as a consequence: all 26 tests pass, with 4 known-bug tests left.

8. Summary & Conclusion

SECTION 8

CHANGE	CATEGORY	PROBLEM	CHANGE MADE	FILES
CM-COR-01	Corrective	one bad cookie terminates the server; missing cookie hangs	send 401s in the auth middleware	1
CM-ADA-01	Adaptive	server cannot start on Node.js ≥ 25	jsonwebtoken 9.0.2 → 9.0.3, Node.js 26 pin	4
CM-PRE-01	Preventive	password, hash and token leak via logs and responses	schema `toJSON` rule, remove credential logs	2
CM-PER-01	Perfective	copied try/catch blocks, error handler ignores status	single error handler, clones removed	3

VERSION	TESTS	PASSING	KNOWN-BUG TESTS
`lab-setup`	20	20	8
`cm-cor-01`	22	22	6
`cm-ada-01`	22	22	6
`cm-pre-01`	25	25	5
`cm-per-01`	26	26	4

Table 3. Test suite at each tagged version (evidence `tests/<tag>.txt`; versions before `cm-ada-01` are run on Node.js 24 because they cannot start on 26).

Residual findings

Comprehension and analysis surfaced more problems than one change per category can address. They are recorded here rather than fixed silently, so that each change above stays small and traceable:

- **Tags are never stored.** `createQuestion` puts `tags` inside the translation instead of on the question, so tag filtering never finds questions created through the API (two `test.failing` tests; confirmed at runtime in §15).
- **Missing fields pass validation.** `field?.trim() === ""` is false for `undefined`, so absent fields reach Mongoose and produce a 500 instead of a 400.
- **Unescaped search.** `?search=` goes straight into `$regex`: special characters cause a 500, and crafted patterns risk regular-expression denial of service.
- **No authorisation on question creation.** `POST /api/questions/add` is open to anonymous clients.
- **Lint findings** (§12): a duplicate `secure` key in the login cookie options makes the intended `NODE_ENV`-dependent setting dead code; two unused variables.
- **Dependencies:** `npm audit` still reports 8 vulnerabilities (2 critical, in `mongoose` and `proxy-addr`; others in `express`, `body-parser`, `cloudinary`, `lodash`, `path-to-regexp`, `qs`). All have fixed versions available.
- **Smaller items:** the login email is logged; a token for a deleted user passes authentication with `req.user = null`; `cors()` with default options cannot serve credentialed cross-origin requests from a separate frontend.

Conclusion

Each category produced a change of a different character: a fault visible to every user (corrective), a failure caused only by the environment moving on (adaptive), risks that had not yet caused harm (preventive), and a structural weakness that was quietly producing wrong status codes (perfective). The five-step process kept each change small: comprehension located the cause, impact analysis bounded what could break, and reverse engineering recovered the design knowledge the change depended on. The test-first approach made every claim checkable: each fix is a known-bug test turning into a passing test, at a tagged commit anyone can check out.

9. Tools Used & References

SECTION 9

TOOL	VERSION	USED FOR
Node.js / npm	26.10.0 and 24.21.0 / 11.19.1	runtime; 24 for versions before CM-ADA-01
MongoDB (Docker)	7.0.45	database for the server and the tests
Jest + supertest	30.5.2 + 7.3.1	tests, V8 code coverage
ESLint / espree	10.13.0 / 11.2.0	lint findings; JavaScript parser for the AST, metrics and slicing tools
jscpd	5.4.2	clone detection
git	—	tags per change, diffs, worktrees for capturing each version
report/tools/*.mjs	this repository	AST dump, metrics, slicer, workload tracer, CPU-profile summary
Chromium (headless)	—	printing this report to PDF

1. ISO/IEC/IEEE 14764:2022, *Software engineering: Software life cycle processes: Maintenance*.
2. Course material: CSE 4802 Lab 1 (Static and Dynamic Analysis; Program Slicing), Lab 3 (Transformation Tools), Lab 6 (Code Clone Detection).
3. K. Tran, K. Nguyen, C. Borcea, N. Phan, "Program Structure-aware Language Models: Targeted Software Testing beyond Textual Semantics", arXiv:2604.17715, 2026 (Lab 2).
4. Node.js deprecations, DEP0030 `SlowBuffer` : https://nodejs.org/api/deprecations.html
5. Express 4 error handling (promises returned by middleware are not handled): https://expressjs.com/en/guide/error-handling.html
6. MongoDB issue SERVER-121912 (Linux 6.19+ incompatibility): https://jira.mongodb.org/browse/SERVER-121912
7. Subject repository: https://github.com/notAvailable73/backend-Nodejs (forked from suffahUniversity/backend-Nodejs).

PART 2 · STATIC & DYNAMIC ANALYSIS AND PROGRAM SLICING

10. Overview & Methodology

SECTION 10

Part 2 applies the analysis techniques from the lab sessions to the same codebase. The lab examples use Python tools; this project is JavaScript, so each technique was mapped to an equivalent for Node.js. Where no suitable tool existed, a small, readable script was written in `report/tools/` so that every definition (what counts as a line, a decision point, a dependency) is explicit and can be explained.

TECHNIQUE (LAB TOOL)	USED HERE	WHY
AST generation (Python `ast.dump`, AST Explorer)	espree + `tools/ast.mjs`	espree is ESLint's parser and produces the standard ESTree AST; the dump format mirrors `ast.dump`
Metrics (radon, SonarQube)	`tools/metrics.mjs` + ESLint recommended rules	size and McCabe complexity with documented counting rules; ESLint for defect-prone patterns
Dynamic analysis: tracing (`sys.settrace`), coverage	Jest with V8 coverage	V8 records executed code natively, without instrumenting the source
Program slicing (IntelliJ "Find usages" / "Analyze data flow")	`tools/slice.mjs`, plus one-test coverage for the dynamic slice	scriptable and repeatable, prints the slice on the source
Loguru, PySnooper	`tools/workload.mjs --trace`	request log plus every MongoDB operation, without editing application code
cProfile, SnakeViz, VizTracer	`node --cpu-prof` + `tools/profile.mjs`	V8's built-in sampling profiler; summary table and icicle chart
Code clone detection (Lab 6)	jscpd 5.4.2	token-based detector with JavaScript support and machine-readable output
Structure-aware test generation (GLMTest, Lab 2)	idea applied by hand	no model was trained; the `test.failing` tests target specific risky branches, which is the paper's goal

Unless stated otherwise, analyses compare the original code (`baseline`, or `lab-setup` where tests are needed; their `src/` is identical) with the final version `cm-per-01`.

11. Static Analysis A: AST Generation

SECTION 11

An Abstract Syntax Tree represents source code as a tree: each node is a construct (a declaration, a call, an identifier) and each edge says which construct contains which. Static analysis tools work on this tree rather than on text. The first statement of the authentication middleware, dumped in the same style as Python's `ast.dump`:

cm-per-01 — node v26.10.0

```
$ node report/tools/ast.mjs dump src/middlewares/auth.middleware.js 6
source  const token = req.cookies.token;

VariableDeclaration(declarations=[VariableDeclarator(id=Identifier(name='token'), init=MemberExpression(object=MemberExpression(object=Ident
ifier(name='req'), property=Identifier(name='cookies')), property=Identifier(name='token')))], kind='const')
```

One statement and its AST.

evidence/part2/ast-dump.txt · tag cm-per-01 (b11cba5) · Node 26.10.0

Reading the output. `const token = req.cookies.token;` is a `VariableDeclaration` of kind `const` with one `VariableDeclarator`. Its `id` is the name being defined (`token`) and its `init` is the value: a `MemberExpression` whose object is another `MemberExpression` (`req.cookies`). Property access nests from the inside out. The slicer in §14 uses exactly this distinction: `id` is a definition, identifiers inside `init` are uses.

cm-per-01 — node v26.10.0

```
$ node report/tools/ast.mjs outline src/middlewares/auth.middleware.js isAuthenticated
ArrowFunctionExpression async  L5
  Identifier req  L5
  Identifier res  L5
  Identifier next  L5
  BlockStatement  L5
    VariableDeclaration const  L6
      VariableDeclarator  L6
        Identifier token  L6
        MemberExpression  L6
          MemberExpression  L6
            Identifier req  L6
            Identifier cookies  L6
          Identifier token  L6
    IfStatement  L8
      UnaryExpression !  L8
        Identifier token  L8
      BlockStatement  L8
        ReturnStatement  L9
          CallExpression  L9
            MemberExpression  L9
              NewExpression  L9
                Identifier ApiError  L9
                Literal 401  L9
                Literal "Unauthorized"  L9
                Literal "No token provided"  L9
              Identifier send  L9
            Identifier res  L9
    TryStatement  L12
      BlockStatement  L12
        VariableDeclaration const  L13
          VariableDeclarator  L13
            Identifier decoded  L13
            CallExpression  L13
              MemberExpression  L13
                Identifier jwt  L13
                Identifier verify  L13
              Identifier token  L13
              MemberExpression  L13
                MemberExpression  L13
                  Identifier process  L13
                  Identifier env  L13
                Identifier JWT_SECRET  L13
        ExpressionStatement  L14
          AssignmentExpression =  L14
            MemberExpression  L14
              Identifier req  L14
              Identifier user  L14
            AwaitExpression  L14
              CallExpression  L14
                MemberExpression  L14
                  CallExpression  L14
                    MemberExpression  L14
                      Identifier User  L14
                      Identifier findById  L14
                    MemberExpression  L14
                      Identifier decoded  L14
                      Identifier id  L14
                  Identifier select  L14
                Literal "-password"  L14
      CatchClause  L15
        Identifier error  L15
        BlockStatement  L15
… 13 line(s) below omitted
```

The whole `isAuthenticated` function as an indented tree (final version). The `IfStatement` and `TryStatement` nodes are the branches discussed in §4.

evidence/part2/ast-outline.txt · tag cm-per-01 (b11cba5) · Node 26.10.0

baseline — node v26.10.0

```
$ node report/tools/ast.mjs stats src
17 files, 2000 AST nodes, 42 node types

Identifier                786 ████████████████████████████
Literal                   186 ███████
MemberExpression          178 ███████
Property                  129 █████
CallExpression            117 ████
ExpressionStatement        68 ███
ObjectExpression           51 ██
BlockStatement             49 ██
VariableDeclaration        48 ██
VariableDeclarator         48 ██
ImportDeclaration          36 ██
NewExpression              24 ▉
ImportSpecifier            21 ▉
ThisExpression             20 ▉
ArrowFunctionExpression    19 ▉
… 27 more node types
```

Node types at `baseline`.

evidence/part2/ast-stats-baseline.txt · tag baseline (dfe8310) · Node 26.10.0

cm-per-01 — node v26.10.0

```
$ node report/tools/ast.mjs stats src
17 files, 1903 AST nodes, 42 node types

Identifier                744 ███████████████████████████
Literal                   178 ███████
MemberExpression          163 ██████
Property                  127 ██████
CallExpression            111 ████
ExpressionStatement        62 ███
ObjectExpression           50 ███
BlockStatement             45 ██
VariableDeclaration        45 ██
VariableDeclarator         45 ██
ImportDeclaration          37 ██
ImportSpecifier            22 ▉
NewExpression              22 ▉
ArrowFunctionExpression    20 ▉
ThisExpression             20 ▉
… 27 more node types
```

Node types at `cm-per-01`.

evidence/part2/ast-stats-final.txt · tag cm-per-01 (b11cba5) · Node 26.10.0

Insight. The tree shrinks by about 100 nodes. Fewer `Identifier`, `MemberExpression` and `NewExpression` nodes reflect the removed catch blocks, each of which built an `ApiError` from `error.statusCode || 500`, `error.errors || error` and `error.message || …`. One extra `ImportDeclaration` is the error handler's new import of `ApiError`.

12. Static Analysis B: Metrics Calculation

SECTION 12

`tools/metrics.mjs` computes, per file, physical lines, source lines (SLOC: lines holding code, not blank and not only comments), comment lines and functions; and per function the McCabe cyclomatic complexity (CC): 1 plus the number of decision points (`if` , `?:` , loops, `case` , `catch` , `&&` , `||` , `??`). CC is the number of independent paths through a function, and so a lower bound on the tests needed to cover it.

baseline — node v26.10.0

```
$ node report/tools/metrics.mjs src
file                                lines  sloc  cmt  fns maxCC
app.js                                 27    19    6    1     1
config/database.js                     12    11    1    1     2
controllers/auth.controller.js         81    63    2    6     8
controllers/question.controller.js     93    75    8    3    11
middlewares/auth.middleware.js         19    16    0    1     3
middlewares/multer.middleware.js       15    12    0    2     1
models/question.model.js               42    37    0    0     -
models/seminar.model.js                39    36    0    0     -
models/user.model.js                   55    47    3    3     2
routes/auth.routes.js                  12     9    0    0     -
routes/protected.routes.js             17    14    0    1     1
routes/question.route.js               13     6    0    0     -
server.js                              31    21    4    2     3
utils/ApiError.js                      22    21    0    2     1
utils/ApiResponse.js                   16    14    0    2     1
utils/asyncHandler.js                   8     6    0    3     1
utils/cloudinary.js                    30    25    1    1     3
TOTAL                                 532   432   25   28    11

Most complex functions (cyclomatic complexity):
  CC 11  getQuestions  (controllers/question.controller.js:38)
  CC  8  loginUser  (controllers/auth.controller.js:27)
  CC  6  createQuestion  (controllers/question.controller.js:6)
  CC  3  registerUser  (controllers/auth.controller.js:6)
  CC  3  isAuthenticated  (middlewares/auth.middleware.js:5)
  CC  3  startServer  (server.js:10)

mean CC 2.18 over 28 functions
```

Metrics at `baseline` .

evidence/part2/metrics-baseline.txt · tag baseline (dfe8310) · Node 26.10.0

cm-per-01 — node v26.10.0

```
$ node report/tools/metrics.mjs src
… 18 line(s) above omitted
TOTAL                                509   410   26   29     7

Most complex functions (cyclomatic complexity):
  CC  7  getQuestions  (controllers/question.controller.js:28)
  CC  4  loginUser  (controllers/auth.controller.js:26)
  CC  3  registerUser  (controllers/auth.controller.js:6)
  CC  3  isAuthenticated  (middlewares/auth.middleware.js:5)
  CC  3  startServer  (server.js:10)
  CC  3  uploadToCloudinary  (utils/cloudinary.js:11)

mean CC 1.76 over 29 functions
```

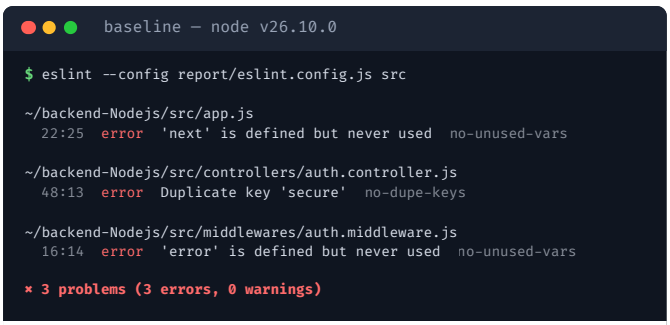
Metrics at `cm-per-01` (per-file table omitted).

evidence/part2/metrics-final.txt · tag cm-per-01 (b11cba5) · Node 26.10.0

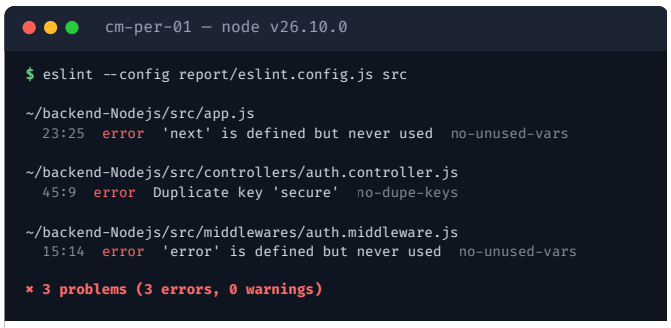
SLOC	MAX CC	MEAN CC	FUNCTIONS
410	**7**	**1.76**	**29**
from 432	from 11	from 2.18	from 28

Insight. Most of the complexity sat in the copied catch blocks: each `||` fallback is a decision point, so removing them cut `getQuestions` from 11 to 7 and `loginUser` from 8 to 4. The function count rises by one: the new `toJSON` transform from CM-PRE-01.

Lint findings



ESLint (recommended rules) at `baseline`.
evidence/part2/eslint-baseline.txt · tag baseline (dfe8310) · Node 26.10.0



The same three findings remain at `cm-per-01`.
evidence/part2/eslint-final.txt · tag cm-per-01 (b11cba5) · Node 26.10.0

Insight. `no-dupe-keys` is a real defect. The login cookie options set `secure: process.env.NODE_ENV === "production"` and then `secure: true`; the second key wins, so the environment-dependent rule the author wrote never takes effect. The two `no-unused-vars` findings are harmless (Express needs the four-argument signature to recognise an error handler). None of the three was in scope for the four changes, so they are listed as residual (§8).

13. Dynamic Analysis: Code Coverage

SECTION 13

Coverage records which code actually ran while the test suite executed: the dynamic counterpart of reading the code. V8 collects it natively; Jest maps it back to source files. All files under `src/` are included, including those no test imports, so that untested code shows up as 0% instead of being silently left out.

lab-setup — node v24.21.0

```
$ npm test -- --coverage --coverageProvider=v8 --collectCoverageFrom='src/**/*.js'
-------------------------|---------|----------|---------|---------|-------------------
File                     | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------------------|---------|----------|---------|---------|-------------------
All files                |   72.74 |    77.14 |   61.53 |   72.74 |
 src                     |   46.55 |    66.66 |       0 |   46.55 |
  app.js                 |     100 |      100 |     100 |     100 |
  server.js              |       0 |        0 |       0 |       0 | 1-31
 src/config              |       0 |        0 |       0 |       0 |
  database.js            |       0 |        0 |       0 |       0 | 1-12
 src/controllers         |   94.25 |    73.68 |     100 |   94.25 |
  auth.controller.js     |   95.06 |    72.22 |     100 |   95.06 | 11-12,32-33
  question.controller.js |   93.54 |       75 |     100 |   93.54 | 12-17
 src/middlewares         |   55.88 |    85.71 |      50 |   55.88 |
  auth.middleware.js     |     100 |      100 |     100 |     100 |
  multer.middleware.js   |       0 |        0 |       0 |       0 | 1-15
 src/models              |   71.32 |    71.42 |   66.66 |   71.32 |
  question.model.js      |     100 |      100 |     100 |     100 |
  seminar.model.js       |       0 |        0 |       0 |       0 | 1-39
  user.model.js          |     100 |       80 |     100 |     100 | 38
 src/routes              |   80.95 |      100 |     100 |   80.95 |
  auth.routes.js         |     100 |      100 |     100 |     100 |
  protected.routes.js    |   52.94 |      100 |     100 |   52.94 | 7-14
  question.route.js      |     100 |      100 |     100 |     100 |
 src/utils               |   60.52 |     90.9 |   83.33 |   60.52 |
  ApiError.js            |     100 |      100 |     100 |     100 |
  ApiResponse.js         |     100 |      100 |     100 |     100 |
  asyncHandler.js        |     100 |      100 |     100 |     100 |
  cloudinary.js          |       0 |        0 |       0 |       0 | 1-30
-------------------------|---------|----------|---------|---------|-------------------
Tests:       20 passed, 20 total
```

Coverage of the baseline suite (tag `lab-setup`).

evidence/part2/coverage-baseline.txt · tag lab-setup (48c5b89) · Node 24.21.0

cm-per-01 — node v26.10.0

```
$ npm test -- --coverage --coverageProvider=v8 --collectCoverageFrom='src/**/*.js'
-------------------------|---------|----------|---------|---------|-------------------
File                     | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------------------|---------|----------|---------|---------|-------------------
All files                |   71.51 |    84.37 |   64.28 |   71.51 |
 src                     |      50 |       80 |       0 |      50 |
  app.js                 |     100 |      100 |     100 |     100 |
  server.js              |       0 |        0 |       0 |       0 | 1-31
 src/config              |       0 |        0 |       0 |       0 |
  database.js            |       0 |        0 |       0 |       0 | 1-12
 src/controllers         |    92.8 |    85.71 |     100 |    92.8 |
  auth.controller.js     |   94.02 |    85.71 |     100 |   94.02 | 10-11,30-31
  question.controller.js |   91.66 |    85.71 |     100 |   91.66 | 11-16
 src/middlewares         |   55.88 |     87.5 |      50 |   55.88 |
  auth.middleware.js     |     100 |      100 |     100 |     100 |
  multer.middleware.js   |       0 |        0 |       0 |       0 | 1-15
 src/models              |   72.91 |       75 |      75 |   72.91 |
  question.model.js      |     100 |      100 |     100 |     100 |
  seminar.model.js       |       0 |        0 |       0 |       0 | 1-39
  user.model.js          |     100 |    83.33 |     100 |     100 | 46
 src/routes              |   80.95 |      100 |     100 |   80.95 |
  auth.routes.js         |     100 |      100 |     100 |     100 |
  protected.routes.js    |   52.94 |      100 |     100 |   52.94 | 7-14
  question.route.js      |     100 |      100 |     100 |     100 |
 src/utils               |   60.52 |     90.9 |   83.33 |   60.52 |
  ApiError.js            |     100 |      100 |     100 |     100 |
  ApiResponse.js         |     100 |      100 |     100 |     100 |
  asyncHandler.js        |     100 |      100 |     100 |     100 |
  cloudinary.js          |       0 |        0 |       0 |       0 | 1-30
-------------------------|---------|----------|---------|---------|-------------------
Tests:       26 passed, 26 total
```

Coverage of the final suite (tag `cm-per-01`).

evidence/part2/coverage-final.txt · tag cm-per-01 (b11cba5) · Node 26.10.0

MEASURE	LAB-SETUP	CM-PER-01
Lines / statements	72.74%	71.51%
Branches	77.14%	84.37%
Functions	61.53%	64.28%

Insight. Line coverage barely moves while branch coverage rises. The removed catch blocks were mostly *covered lines* but *uncovered branches* (the `||` fallbacks rarely ran), so deleting them lowers the line denominator and raises the branch ratio. Percentages compare different denominators and should be read as "how much of this version ran", not as progress on a fixed scale. The 0% files are the real finding: `server.js`, `config/database.js`, `multer.middleware.js`, `cloudinary.js` and `seminar.model.js` are never loaded by any test. The last three are not used by any route at all: candidates for removal or for the next feature. `protected.routes.js` lines 7–14 show that no test reaches the protected route's success path.

14. Program Slicing (Static & Dynamic)

SECTION 14

A **slice** is the set of statements that can affect the value of a variable at a chosen point, the *slicing criterion*. A **static** slice considers every possible execution; a **dynamic** slice only the statements that affected the value in one particular execution, so it is usually smaller. `tools/slice.mjs` computes backward slices inside a function by following two kinds of dependency:

- **Data:** a statement that uses variable *v* depends on earlier statements that define *v* (for `query.tags = …`, the object `query` counts as both used and defined).
- **Control:** a statement depends on the `if` or loop test that decides whether it runs, including an earlier `if (…) { return/throw }` that can skip it.

Criterion: the MongoDB query object `query` at line 47 of `getQuestions`, where it is passed to `Question.find`. This is the question a maintainer asks when a search returns the wrong questions: *which code decides what is searched for?*

cm-per-01 — node v26.10.0

```
$ node report/tools/slice.mjs src/controllers/question.controller.js getQuestions 47 --var query
Static backward slice of getQuestions (question.controller.js)
criterion: line 47, variables {query}
legend: ▶ criterion  ● in slice

   28  const getQuestions = asyncHandler(async (req, res) => {
   29    console.log("Fetching questions");
   30
●  31    const { tags, search } = req.query;
   32
●  33    let query = {};
   34
   35    // Handle tag filtering
●  36    if (tags) {
●  37      const tagArray = Array.isArray(tags) ? tags : tags.split(',');
●  38      query.tags = { $in: tagArray };
   39    }
   40
   41    // Handle search filtering
●  42    if (search) {
●  43      query['translations.title'] = { $regex: search, $options: 'i' }; // Case-insensitive search in title
   44    }
   45
   46    // Fetch questions
▶  47    const questions = await Question.find(query)
▶  48      .select('-__v') // Exclude version key
▶  49      .sort({ createdAt: -1 }); // Sort by latest
   50
   51    // If no questions found
   52    if (!questions || questions.length === 0) {
   53      throw new ApiError(
   54        404,
   55        [],
   56        search ? `No questions found matching "${search}"` : 'No questions found'
   57      );
   58    }
   59
   60    // Return response
   61    return new ApiResponse(
   62      200,
   63      {
   64        count: questions.length,
   65        questions
   66      },
   67      "Questions fetched successfully"
   68    ).send(res);
   69  });

8 of 12 statements/predicates in the slice; depends on parameter(s) req
```

Static slice: everything that may shape the query.

evidence/part2/slice-static.txt · tag cm-per-01 (b11cba5) · Node 26.10.0

The static slice keeps both filters, the tag filter (lines 36–38) and the title search (42–43), and the destructuring of `req.query` that feeds them; logging, the 404 check and the response are excluded. For the dynamic slice, one test, "searches titles case-insensitively" (`GET /api/questions?search=PRAYER`), was run alone with coverage, and the slicer was restricted to the statements that executed:

cm-per-01 — node v26.10.0

```
$ npm test -- tests/questions.test.js -t 'searches titles' --coverage
Tests:       9 skipped, 1 passed, 10 total
```

Running the single test that defines the execution.

evidence/part2/slice-run.txt · tag cm-per-01 (b11cba5) · Node 26.10.0

cm-per-01 — node v26.10.0

```
$ node report/tools/slice.mjs src/controllers/question.controller.js getQuestions 47 --var query --coverage coverage/coverage-final.json
Dynamic backward slice of getQuestions (question.controller.js)
criterion: line 47, variables {query}
legend: ▶ criterion  ● in slice  · did not execute

   28  const getQuestions = asyncHandler(async (req, res) => {
   29    console.log("Fetching questions");
   30
●  31    const { tags, search } = req.query;
   32
●  33    let query = {};
   34
   35    // Handle tag filtering
   36    if (tags) {
·  37      const tagArray = Array.isArray(tags) ? tags : tags.split(',');
·  38      query.tags = { $in: tagArray };
·  39    }
   40
   41    // Handle search filtering
●  42    if (search) {
●  43      query['translations.title'] = { $regex: search, $options: 'i' }; // Case-insensitive search in title
   44    }
   45
   46    // Fetch questions
▶  47    const questions = await Question.find(query)
▶  48      .select('-__v') // Exclude version key
▶  49      .sort({ createdAt: -1 }); // Sort by latest
   50
… 21 line(s) below omitted
```

Dynamic slice for that execution (· marks lines that did not run).

evidence/part2/slice-dynamic.txt · tag cm-per-01 (b11cba5) · Node 26.10.0

Insight. The dynamic slice drops the tag branch completely. The `if (tags)` test *did* run (it evaluated to false) but is not in the slice, because nothing it controls contributed to `query`. That is the defining difference from coverage: coverage says what ran, the slice says what *mattered*. In maintenance terms, a bug report about title search can be investigated in five statements instead of twelve. The same slice explains the residual tags bug: `query.tags` filters the top-level `tags` field, which the static slice of `createQuestion` would show is never written.

Limitations: the slicer works inside one function, does not do kill analysis (every earlier definition counts), and treats a method call `obj.m(…)` as possibly modifying `obj`. These over-approximate, so the slices may be larger than necessary but do not miss statements in code of this shape.

15. Runtime Analysis: Tracing & Profiling

TRANSFORMATION TOOLS (LAB 3)

The lab's transformation tools turn a running program into a readable form: a log (Loguru), a line-by-line trace (PySnooper), a timeline or a chart of where time goes (VizTracer, cProfile + SnakeViz). For this server, the useful unit of execution is an HTTP request and the database work it causes. `tools/workload.mjs` loads the unmodified application, wraps it in a request logger, enables Mongoose's query hook, and replays a scripted session. Passwords' hashes and tokens are masked in the output.

cm-per-01 — node v26.10.0

```
$ node report/tools/workload.mjs . --trace
2026-10-26 12:43:34.117 | SUCCESS | ── register a new user
2026-10-26 12:43:34.152 | DEBUG   |     db.users.findOne({"email":"trace@example.com"}, {})
2026-10-26 12:43:34.154 | DEBUG   |     db.users.createIndex({"email":1}, {"unique":true,"background":true})
2026-10-26 12:43:34.232 | DEBUG   |     db.users.insertOne({"name":"Trace User","email":"trace@example.com","password":"<bcrypt hash>","role":"user","ask
edQuestions":[],"_id":"6adf4af6ae67bd23352b3af1","createdAt":"2026-10-26T12:43:34.160Z","updatedAt":"2026-10-26T12:43:34.160Z","__v":0}, {})
2026-10-26 12:43:34.239 | INFO    | POST /api/auth/register → 201 (94.5 ms)
2026-10-26 12:43:34.243 | SUCCESS | ── log in with a wrong password
2026-10-26 12:43:34.246 | DEBUG   |     db.users.findOne({"email":"trace@example.com"}, {})
2026-10-26 12:43:34.314 | WARNING | POST /api/auth/login → 401 (69.0 ms)
2026-10-26 12:43:34.315 | SUCCESS | ── log in
2026-10-26 12:43:34.317 | DEBUG   |     db.users.findOne({"email":"trace@example.com"}, {})
2026-10-26 12:43:34.386 | INFO    | POST /api/auth/login → 200 (69.4 ms)
2026-10-26 12:43:34.387 | SUCCESS | ── read the profile with the session cookie
2026-10-26 12:43:34.390 | DEBUG   |     db.users.findOne({"_id":"6adf4af6ae67bd23352b3af1"}, {"projection":{"password":0}})
2026-10-26 12:43:34.392 | INFO    | GET /api/auth/profile → 200 (4.5 ms)
2026-10-26 12:43:34.393 | SUCCESS | ── read the profile with a forged token
2026-10-26 12:43:34.394 | WARNING | GET /api/auth/profile → 401 (0.5 ms)
2026-10-26 12:43:34.394 | SUCCESS | ── add a question tagged 'fiqh'
2026-10-26 12:43:34.398 | DEBUG   |     db.questions.insertOne({"translations":[{"language":"en","title":"How to perform wudu?","description":"Steps of a
blution","answer":[],"_id":"6adf4af6ae67bd23352b3af7"}],"isFeatured":false,"tags":[],"_id":"6adf4af6ae67bd23352b3af6","createdAt":"2026-10-26T12:43:34.39
8Z","updatedAt":"2026-10-26T12:43:34.398Z","__v":0}, {})
2026-10-26 12:43:34.399 | INFO    | POST /api/questions/add → 201 (4.7 ms)
2026-10-26 12:43:34.400 | SUCCESS | ── list questions tagged 'fiqh'
2026-10-26 12:43:34.402 | DEBUG   |     db.questions.find({"tags":{"$in":["fiqh"]}}, {"sort":{"createdAt":-1},"projection":{"__v":0}})
2026-10-26 12:43:34.403 | WARNING | GET /api/questions?tags=fiqh → 404 (2.4 ms)
2026-10-26 12:43:34.404 | SUCCESS | ── search question titles for 'wudu'
2026-10-26 12:43:34.405 | DEBUG   |     db.questions.find({"translations.title":{"$regex":"wudu","$options":"i"}}, {"sort":{"createdAt":-1},"projection":
{"__v":0}})
2026-10-26 12:43:34.408 | INFO    | GET /api/questions?search=wudu → 200 (3.4 ms)
```

Request/query trace of the final version, in Loguru's `time | level | message` format.

evidence/part2/trace.txt · tag cm-per-01 (b11cba5) · Node 26.10.0

Insights from the trace.

- Registration costs three database operations, including a `createIndex` issued by Mongoose on the first write to `users`; the unique e-mail index is created lazily, not by a migration.
- Login takes ~70 ms against ~2–5 ms for every other request, all of it before the database reply is used: password hashing (see the profile below).
- The profile lookup excludes the password at the query level (`projection: {password: 0}`), and the forged token is rejected in under a millisecond with **no** database query: the CM-COR-01 path.
- The `insertOne` for the new question stores `"tags": []`, and the next request's tag query returns 404. This is the residual tags bug observed at runtime, not just inferred from the code.

CPU profiling

V8's sampling profiler (`node --cpu-prof`) records the call stack every millisecond. A workload of 30 rounds of login, profile, add question and search was profiled; `tools/profile.mjs` summarises the profile by package and function (the cProfile view) and draws an icicle chart of the call tree (the SnakeViz view).

cm-per-01 — node v26.10.0

```
$ node --cpu-prof report/tools/workload.mjs . --iterations 30
workload finished: 30 iterations of login, profile, add question, search

$ node report/tools/profile.mjs workload.cpuprofile --svg icicle.svg
sampled 2797.4 ms; idle 59.1 ms; busy (CPU) 2738.3 ms

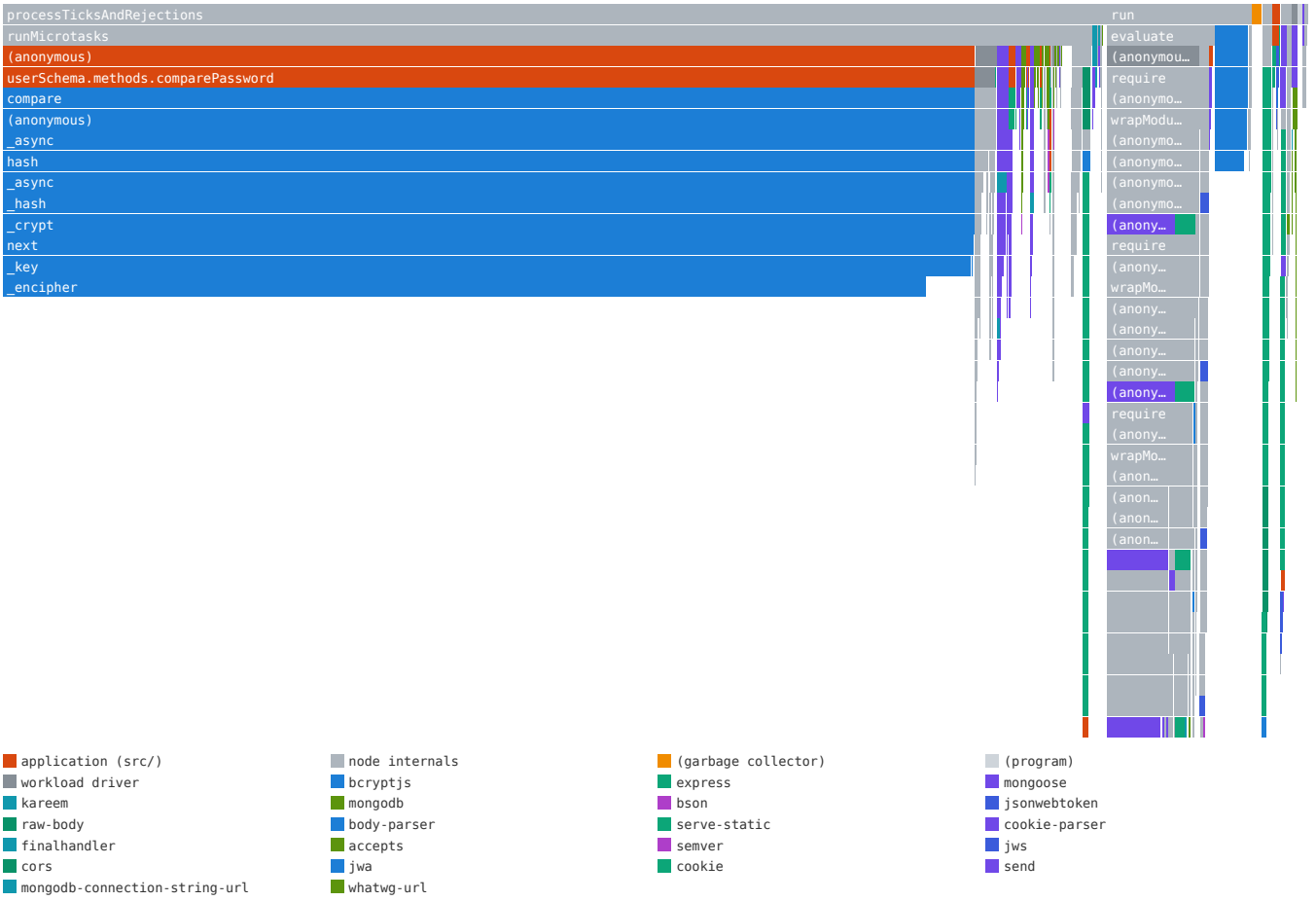
CPU time by package (self time)
  bcryptjs                  2099.5 ms  76.7%  ████████████████████████████
  node internals             377.0 ms  13.8%  ██████
  mongoose                    92.0 ms   3.4%  ▏
  mongodb                     52.5 ms   1.9%  ▏
  (garbage collector)         21.3 ms   0.8%
  express                     11.7 ms   0.4%
  bson                        11.1 ms   0.4%
  (program)                    9.4 ms   0.3%

Top functions by self time
       self      %  function  location
   1980.9 ms  72.3%  _encipher  bcryptjs/index.js:743
     98.5 ms   3.6%  _key  bcryptjs/index.js:881
     67.3 ms   2.5%  wrapSafe  node:internal/modules/cjs/loader:1846
     21.3 ms   0.8%  (garbage collector)
     18.2 ms   0.7%  internalModuleStat
     15.9 ms   0.6%  _streamtoword  bcryptjs/index.js:868
     11.6 ms   0.4%  compileForInternalLoader  node:internal/bootstrap/realm:407
      9.4 ms   0.3%  (program)
      8.5 ms   0.3%  (anonymous)  node:internal/modules/cjs/loader:784
      7.4 ms   0.3%  runMicrotasks
      6.8 ms   0.2%  normalizeString  node:path:92
      6.6 ms   0.2%  readFileUtf8

Application functions by total time (including callees)
   2048.8 ms  74.8%  (anonymous)  src/controllers/auth.controller.js:26
   2031.7 ms  74.2%  userSchema.methods.comparePassword  src/models/user.model.js:59
     32.9 ms   1.2%  send  src/utils/ApiResponse.js:8
     17.0 ms   0.6%  (anonymous)  src/utils/asyncHandler.js:2
     17.0 ms   0.6%  (anonymous)  src/controllers/question.controller.js:6
     15.8 ms   0.6%  (anonymous)  src/controllers/question.controller.js:28
```

Profile summary of the final version.

evidence/part2/profile.txt · tag cm-per-01 (b11cba5) · Node 26.10.0



Icicle chart of the CPU profile. Each box is a function in a call path; its width is the time spent in it and its callees; callees sit beneath their caller. Orange: application code (`src/`).

evidence/part2/profile-icicle.svg

Insight. 76.7% of CPU time is spent inside `bcryptjs`, and `comparePassword`, called once per login, accounts for 74.2% of the whole workload. The application's own code is negligible. This is by design: bcrypt with cost factor 10 is *meant* to be slow, to make stolen hashes expensive to crack. It is not a defect, but it is a capacity limit worth knowing. Login throughput on one core is bounded at roughly a dozen per second, and `bcryptjs` is a pure-JavaScript implementation that blocks the event loop while hashing. A native binding (`bcrypt`) or moving hashing to a worker thread would be the perfective options if login load grew.

16. Code Clone Detection

LAB 6

Tool overview. jscpd tokenises each file, slides a window over the token stream, and hashes each window (Rabin–Karp); windows with equal hashes in different places are reported as clones once they reach the minimum size. Because it compares tokens, not characters, formatting differences do not hide a clone. It was chosen for its JavaScript support, its precise locations, and its JSON output (used for the numbers below). Thresholds: at least 5 lines and 30 tokens.

Method. jscpd was run on `src/` at `baseline`, the clones were inspected, the refactoring was applied (CM-PER-01), and the same command was run on `cm-per-01`.

baseline — node v26.10.0

```
$ jscpd --min-lines 5 --min-tokens 30 src
Clone found (javascript)
 - controllers/auth.controller.js [52:72 - 59:27] (8 lines, 42 tokens)
   controllers/question.controller.js [26:5 - 32:23]
Clone found (javascript)
 - controllers/auth.controller.js [52:72 - 59:27] (8 lines, 42 tokens)
   controllers/question.controller.js [79:5 - 86:23]
Clone found (javascript)
 - models/question.model.js [3:33 - 9:7] (7 lines, 33 tokens)
   models/seminar.model.js [3:32 - 9:7]
```

Format	Files analyzed	Total lines	Total tokens	Clones found	Duplicated lines	Duplicated tokens
javascript	17	532	2615	3	23 (4.32%)	117 (4.47%)
Total:	17	532	2615	3	23 (4.32%)	117 (4.47%)

```
Found 3 clones.
time: 3.640ms
```

Results at `baseline`.

evidence/part2/jscpd-baseline.txt · tag baseline (dfe8310) · Node 26.10.0

Analysis.

CLONE	LOCATIONS	TYPE	IMPACT
catch-and-resend block	auth.controller.js (loginUser) ↔ question.controller.js (createQuestion, getQuestions)	Type 1 (exact copy) up to identifiers	**Harmful**: behaviour depends on copying it, and a missing copy (registerUser) already causes wrong status codes
translation `language` field	question.model.js ↔ seminar.model.js	Type 1	Low: two schemas declaring the same field; a change to the language list must be made twice

Proposed solutions. For the catch block: centralise error handling in the error-handling middleware and delete the copies. This was implemented as CM-PER-01. For the schema field: extract a shared `languageField` definition (e.g. `models/fields.js`) used by both translation schemas; left as a recommendation, since the seminar model has no routes yet.

cm-per-01 — node v26.10.0

```
$ jscpd --min-lines 5 --min-tokens 30 src
Clone found (javascript)
 - models/question.model.js [3:33 - 9:7] (7 lines, 33 tokens)
   models/seminar.model.js [3:32 - 9:7]
```

Format	Files analyzed	Total lines	Total tokens	Clones found	Duplicated lines	Duplicated tokens
javascript	17	509	2503	1	7 (1.38%)	33 (1.32%)
Total:	17	509	2503	1	7 (1.38%)	33 (1.32%)

```
Found 1 clones.
time: 3.567ms
```

Results at `cm-per-01`.

evidence/part2/jscpd-final.txt · tag cm-per-01 (b11cba5) · Node 26.10.0

Conclusion. Duplicated lines fall from 23 (4.32%) to 7 (1.38%). The lesson from this codebase is that clone count alone understates the risk: the harmful clone was only 8 lines, but it encoded a rule that every controller had to follow by hand.

17. Summary & Caveats

SECTION 17

MEASURE	BEFORE	AFTER	SOURCE
Tests (passing)	20	26	Jest
Known-bug tests	8	4	test files
Source lines (SLOC)	432	410	metrics.mjs
Max / mean cyclomatic complexity	11 / 2.18	7 / 1.76	metrics.mjs
Branch coverage	77.14%	84.37%	Jest / V8
Clones (duplicated lines)	3 (23)	1 (7)	jscpd
ESLint findings	3	3	ESLint
npm audit vulnerabilities	9	8	npm

Caveats.

- Coverage shows what ran, not that it is correct; and percentages of two versions have different denominators (§13).
- Cyclomatic complexity counts decision points; it says nothing about how hard the code is to read.
- The slicer is an approximation (§14); results were checked by hand against the source.
- Profile numbers depend on the machine and the workload; the shares (bcrypt dominates) are stable, the milliseconds are not.
- jscpd only finds clones above its thresholds and does not find semantic (Type 4) clones.
- Timestamps, ids and timings in the terminal panes change on every regeneration; the conclusions do not.

Appendix A. Reproduction

REGENERATE EVERYTHING

Everything in this report, including every terminal pane, every number and the PDF, is produced by two scripts from the tagged commits. Requirements: git, Docker, nvm with Node.js 24.21.0 and 26.10.0, Chromium or Chrome, and poppler-utils for the page numbers in the contents.

```
git clone https://github.com/notAvailable73/backend-Nodejs.git && cd backend-Nodejs
git checkout lab/maintenance
nvm install 24.21.0 && nvm install 26.10.0 && nvm use
npm ci
npm run report            # = report:capture (runs every command) + report:build (HTML + PDF)

npm run report:capture -- --only cor/     # re-capture one section
npm run report:build                      # rebuild the PDF after editing report/template/report.html
```

`report:capture` checks out each tag into its own git worktree (under `~/.cache/jamiya-report/`), runs `npm ci` there, starts MongoDB with Docker Compose, and runs each command below, saving its output to `report/evidence/<id>.txt` and the numbers quoted in the text to `report/evidence/values.json`. `report:build` fills the template with them and prints the PDF with headless Chromium.

Evidence index

EVIDENCE ID	TAG	NODE	COMMAND	EXIT
ada/boot-after	cm-ada-01	26.10.0	report/demos/boot.sh	0
ada/boot-before	cm-cor-01	26.10.0	report/demos/boot.sh	0
ada/deps-after	cm-ada-01	26.10.0	npm ls buffer-equal-constant-time	0
ada/deps-before	cm-cor-01	26.10.0	npm ls buffer-equal-constant-time	0
ada/diff	—	26.10.0	git diff cm-cor-01 cm-ada-01 -- package.json .nvmrc …	0
ada/jwa	cm-ada-01	26.10.0	grep -n -A9 '^var timingSafeEqual' node_modules/jwa/index.js	0
ada/slowbuffer	cm-cor-01	26.10.0	grep -n SlowBuffer node_modules/buffer-equal-constant-time/index.js	0
ada/tests-node24	cm-ada-01	24.21.0	node --version && npm test	0
ada/tests-node26	cm-ada-01	26.10.0	node --version && npm test	0
cor/demo-after	cm-cor-01	24.21.0	report/demos/auth-requests.sh	0
cor/demo-before	lab-setup	24.21.0	report/demos/auth-requests.sh	0
cor/diff	—	26.10.0	git diff lab-setup cm-cor-01 -- src	0
cor/tests	cm-cor-01	24.21.0	npm test -- --verbose tests/auth.test.js -t 'GET /api/auth/profile'	0
part2/ast-dump	cm-per-01	26.10.0	node report/tools/ast.mjs dump src/middlewares/auth.middleware.js 6	0
part2/ast-outline	cm-per-01	26.10.0	node report/tools/ast.mjs outline src/middlewares/auth.middleware.js isAuthenticated	0
part2/ast-stats-baseline	baseline	26.10.0	node report/tools/ast.mjs stats src	0
part2/ast-stats-final	cm-per-01	26.10.0	node report/tools/ast.mjs stats src	0
part2/audit-baseline	lab-setup	26.10.0	npm audit --omit=dev	0
part2/audit-final	cm-per-01	26.10.0	npm audit --omit=dev	0
part2/coverage-baseline	lab-setup	24.21.0	npm test -- --coverage --coverageProvider=v8 --collectCoverageFrom='src/**/*.js'	0
part2/coverage-final	cm-per-01	26.10.0	npm test -- --coverage --coverageProvider=v8 --collectCoverageFrom='src/**/*.js'	0
part2/eslint-baseline	baseline	26.10.0	eslint --config report/eslint.config.js src	1
part2/eslint-final	cm-per-01	26.10.0	eslint --config report/eslint.config.js src	1

EVIDENCE ID	TAG	NODE	COMMAND	EXIT
part2/jscpd-baseline	baseline	26.10.0	jscpd --min-lines 5 --min-tokens 30 src	0
part2/jscpd-final	cm-per-01	26.10.0	jscpd --min-lines 5 --min-tokens 30 src	0
part2/metrics-baseline	baseline	26.10.0	node report/tools/metrics.mjs src	0
part2/metrics-final	cm-per-01	26.10.0	node report/tools/metrics.mjs src	0
part2/profile	cm-per-01	26.10.0	node --cpu-prof report/tools/workload.mjs . --iterations 30 …	0
part2/slice-dynamic	cm-per-01	26.10.0	node report/tools/slice.mjs src/controllers/question.controller.js getQuestions 47 --var query --coverage coverage/coverage-final.json	0
part2/slice-run	cm-per-01	26.10.0	npm test -- tests/questions.test.js -t 'searches titles' --coverage --coverageReporters=json --collectCoverageFrom=src/controllers/question.controller.js	0
part2/slice-static	cm-per-01	26.10.0	node report/tools/slice.mjs src/controllers/question.controller.js getQuestions 47 --var query	0
part2/trace	cm-per-01	26.10.0	node report/tools/workload.mjs . --trace	0
per/diff	—	26.10.0	git diff -w cm-pre-01 cm-per-01 -- src	0
per/diff-stat	—	26.10.0	git diff --stat cm-pre-01 cm-per-01 -- src	0
per/jscpd-after	cm-per-01	26.10.0	jscpd --min-lines 5 --min-tokens 30 src	0
per/jscpd-before	cm-pre-01	26.10.0	jscpd --min-lines 5 --min-tokens 30 src	0
per/tests-after	cm-per-01	26.10.0	npm test	0
per/tests-before	cm-per-01	26.10.0	npm test -- --silent	1
pre/demo-after	cm-pre-01	26.10.0	report/demos/credentials.sh	0
pre/demo-before	cm-ada-01	26.10.0	report/demos/credentials.sh	0
pre/diff	—	26.10.0	git diff cm-ada-01 cm-pre-01 -- src	0
pre/tests-after	cm-pre-01	26.10.0	npm test -- --verbose tests/auth.test.js -t 'credential exposure'	0
pre/tests-before	cm-pre-01	26.10.0	npm test -- --silent --verbose tests/auth.test.js -t 'credential exposure'	1
setup/env	—	26.10.0	uname -sr …	0
setup/history	baseline	26.10.0	git log --format='%h %ad %s' --date=short # at tag baseline	0
setup/mongo8-kernel	—	26.10.0	docker run --rm mongo:8.0.34	0
setup/npm-ci	lab-setup	26.10.0	npm ci	0
setup/src-unchanged	—	26.10.0	git diff --stat baseline lab-setup -- src	0
setup/structure	baseline	—	git ls-tree -r --name-only baseline # drawn as a tree	0
setup/tests	lab-setup	24.21.0	npm test -- --verbose	0
tests/cm-ada-01	cm-ada-01	26.10.0	node --version && npm test	0
tests/cm-cor-01	cm-cor-01	24.21.0	node --version && npm test	0
tests/cm-per-01	cm-per-01	26.10.0	node --version && npm test	0
tests/cm-pre-01	cm-pre-01	26.10.0	node --version && npm test	0
tests/lab-setup	lab-setup	24.21.0	node --version && npm test	0